

Nama : Fahmi Abdullah

Kelas : TI-25-PA2

NPM : 252310057

<https://github.com/252310057-boop/algoritma-semester2.git>

Hashing adalah proses mengubah suatu data atau nilai kunci (*key*) berukuran berapa pun menjadi sebuah nilai acak berukuran tetap (*fixed size*) yang biasanya berupa indeks angka atau kode alfanumerik.

Tujuan hashing adalah untuk mengolah dan mengamankan data agar proses akses/pencarian data menjadi lebih cepat serta integritas dan keamanan data tetap terjaga

Hashing menjadi berapa bagian?

1. Open Addressing

Pada metode ini seluruh data disimpan langsung di dalam tabel hash.

Jika terjadi collision, maka sistem akan mencari slot kosong lainnya menggunakan aturan tertentu.

2. Separate Chaining (Closed Addressing)

Pada metode ini setiap indeks tabel menyimpan sebuah linked list.

Jika terjadi collision, data baru dimasukkan ke linked list pada indeks tersebut.

Collision adalah keadaan ketika dua atau lebih data menghasilkan nilai hash yang sama, sehingga ingin menempati indeks yang sama pada tabel hash.

```

1  #include <iostream>
2  #include <string>
3  #include <vector>
4
5  using namespace std;
6
7  // Struktur untuk menyimpan data inventori
8  struct InventoryItem {
9      int key;
10     string name;
11     bool isOccupied = false;
12 };
13
14 // Fungsi untuk mengecek apakah sebuah angka adalah bilangan prima
15 bool isPrime(int n) {
16     if (n <= 1) return false;
17     for (int i = 2; i * i <= n; i++) {
18         if (n % i == 0) return false;
19     }
20     return true;
21 }
22
23 // Fungsi untuk mencari bilangan prima atas
24 int getNextPrime(int n) {
25     int prime = n + 1;
26     while (!isPrime(prime)) {
27         prime++;
28     }
29     return prime;
30 }
31
32 int main() {
33     int inputSize;
34     cout << "Masukkan jumlah data: ";
35     cin >> inputSize;
36
37     // Menentukan ukuran tabel dengan bilangan prima atas
38     int hdt_boundary = getNextPrime(inputSize);

```

```

39 cout << "Jumlah data telah diatur ulang menjadi " << hdt_boundary << " dengan prima atas." << endl << endl;
40
41 vector<InventoryItem> table(hdt_boundary);
42 int choice;
43
44 do {
45     cout << "- Inventori Manajemen Sederhana -" << endl;
46     cout << "(1) Insert Item" << endl;
47     cout << "(2) Hapus Item" << endl;
48     cout << "(3) Lihat Item" << endl;
49     cout << "(0) Keluar" << endl;
50     cout << "Pilihan: ";
51     cin >> choice;
52
53     if (choice == 1) {
54         string name;
55         int key;
56         cout << "Masukkan item yang ingin disimpan: ";
57         cin.ignore();
58         getline(cin, name);
59         cout << "Masukkan key (berupa angka) untuk hash: ";
60         cin >> key;
61
62         int i = 0;
63         bool inserted = false;
64
65         while (!inserted && i < hdt_boundary) {
66             int hashIndex = (key + i) % hdt_boundary;
67             if (!table[hashIndex].isOccupied) {
68                 table[hashIndex].key = key;
69                 table[hashIndex].name = name;
70                 table[hashIndex].isOccupied = true;
71                 inserted = true;
72                 cout << "Data di insert dengan i: " << i << endl;
73                 cout << "Item berhasil disimpan." << endl;
74             } else {
75                 i++;
76             }
77         }
78
79         if (!inserted) {
80             cout << "Tabel hash sudah penuh!" << endl;
81         }
82         cout << endl;
83
84     } else if (choice == 2) {
85         int key;
86         cout << "Masukkan key (berupa angka) untuk menghapus item: ";
87         cin >> key;
88
89         bool found = false;
90         for (int i = 0; i < hdt_boundary; i++) {
91             int hashIndex = (key + i) % hdt_boundary;
92             if (table[hashIndex].isOccupied && table[hashIndex].key == key) {
93                 table[hashIndex].isOccupied = false;
94                 cout << "Item dengan key " << key << " berhasil dihapus." << endl;
95                 found = true;
96                 break;
97             }
98         }
99         if (!found) {
100             cout << "Item dengan key tersebut tidak ditemukan." << endl;
101         }
102         cout << endl;
103
104     } else if (choice == 3) {
105         int key, i;
106         cout << "Masukkan key (berupa angka) untuk hash melihat item: ";
107         cin >> key;
108         cout << "Masukkan iterator (i) untuk melihat item: ";
109         cin >> i;
110
111         int hashIndex = (key + i) % hdt_boundary;
112         if (table[hashIndex].isOccupied && table[hashIndex].key == key) {
113             cout << "Item dengan key = " << key << " dan i = " << i << ": " << table[hashIndex].name << endl;
114         } else {

```

```

115         cout << "Item tidak ditemukan pada posisi tersebut." << endl;
116     }
117     cout << endl;
118 }
119
120 } while (choice != 0);
121
122 cout << "Terima kasih telah menggunakan sistem inventori!" << endl;
123 return 0;
124 }

```

Masukkan jumlah data: 5

Jumlah data telah diatur ulang menjadi 7 dengan prima atas.

- Inventori Manajemen Sederhana -

(1) Insert Item

(2) Hapus Item

(3) Lihat Item

(0) Keluar

Pilihan: 1

Masukkan item yang ingin disimpan: mouse

Masukkan key (berupa angka) untuk hash: 150

Data di insert dengan i: 0

Item berhasil disimpan.

- Inventori Manajemen Sederhana -

(1) Insert Item

(2) Hapus Item

(3) Lihat Item

(0) Keluar

Pilihan: 3

Masukkan key (berupa angka) untuk hash melihat item: keyboard

Masukkan iterator (i) untuk melihat item: Item tidak ditemukan pada posisi tersebut.